CRISPE Coding Prompt: Task Tracker API

CAPACITY

Act as a Senior Backend Software Engineer with expertise in Node.js, Express.js, REST API design, SQL databases, security, and backend testing.

RECIPIENT

The target audience is a junior QA and developer team. Write code that is easy to understand, maintain, test, and debug.

INSTRUCTION

Implement the following REST API route controllers for the Task Tracker API:

1. Create Task

- Method: `POST`
- Endpoint: `/api/v1/tasks`
- Extract `title`, `description`, and `due_date` from the request body.
- Validate required fields, ensuring `title` is provided and is not blank.
- Associate the task with the authenticated user's ID.
- Insert the task into the `tasks` table using a parameterized raw SQL query.
- Use the default status `Pending` when no status is provided.
- Return the created task and an appropriate HTTP status code.

2. Get Task by ID

- Method: `GET`
- Endpoint: `/api/v1/tasks/:id`
- Extract the task ID from the URL parameters.
- Validate that the task ID is a valid positive integer.
- Retrieve the task using a parameterized raw SQL query.
- Ensure the authenticated user is authorized to access the task.
- Return the task details if found; otherwise, return an appropriate error response.

STYLE

- Follow DRY (Don't Repeat Yourself) and SOLID design principles.
- Use a clean separation between route definitions, controllers, database access, and error handling.
- Use `async/await` for database operations.
- Use parameterized SQL queries to prevent SQL injection.
- Return consistent JSON responses.
- Use appropriate HTTP status codes, including `201`, `200`, `400`, `401`, `403` or `404` as applicable, and `500` for unexpected server errors.
- Do not expose database credentials, SQL internals, stack traces, or sensitive information in client-facing errors.
- Keep the implementation beginner-friendly with clear naming and modular functions.

PARAMETERS

- Use Node.js with Express.js and PostgreSQL.
- Use the `pg` package for database connectivity.
- Do not use an ORM.
- Match the database schema defined in Step 3:
  - `users`: `id`, `name`, `email`, `password_hash`, `created_at`
  - `tasks`: `id`, `user_id`, `title`, `description`, `status`, `due_date`, `created_at`
- Use the authenticated user's ID from trusted authentication middleware, such as `req.user.id`. Do not accept the owner ID directly from the request body.
- Use raw SQL with positional parameters such as `$1`, `$2`, and `$3`.
- Assume authentication middleware has already been configured, but clearly document this dependency.
- Use the following SQL query patterns:
  - INSERT: `INSERT INTO tasks (user_id, title, description, due_date) VALUES ($1, $2, $3, $4) RETURNING *`
  - SELECT: `SELECT * FROM tasks WHERE id = $1 AND user_id = $2`
- Respect the database constraints and default task status.

EVALUATION

- Provide the complete, runnable example, including imports, database connection setup, route definitions, controller logic, and centralized error handling.
- Include inline comments explaining how request body parameters, URL parameters, and authenticated user information are parsed.
- Explain each SQL query and its parameter bindings.

- Validate inputs before executing SQL queries.
- Handle invalid IDs, missing required fields, nonexistent tasks, unauthorized requests, and database errors.
- Avoid SQL string concatenation with user-provided input.
- Include example success and error responses for both endpoints.
- Briefly explain how the implementation satisfies DRY, SOLID, and REST API design principles.
- Do not use an ORM or generate a new database schema.